作业一：文本分类实现 —— 实验报告

姓名：许可　　学号：2513590

一、任务简介

本实验以 New York Times（NYT）新闻分类为任务，系统比较三种文本表示方法——传统词袋模型（Bag of Words）、词向量（Word Embedding / Word2Vec）以及预训练语言模型（BERT）——在文本分类中的性能。全部方法在统一的 NYT Test Set 上使用 Accuracy 与 Macro-F1 两个指标进行评价，共完成 7 组实验。

二、数据集与预处理

2.1 NYT 数据集

NYT 数据集包含 11,519 条新闻，每条包含两列：`text`（新闻正文）与 `label`（新闻类别）。共有 3 个类别，各类别样本数如下表所示：

类别	样本数	占比
sports	8,639	75.0%
politics	1,451	12.6%
business	1,429	12.4%

可见类别分布极不均衡，sports 类占约四分之三。这意味着仅靠 Accuracy 不足以全面评价模型——一个“全部预测为 sports”的懒惰分类器也能获得约 75% 的准确率。因此 Macro-F1 是更关键的评价指标，

2.2 AG News 数据集

AG News 数据集共 90,000 条新闻文本（单列 `text`），在本实验中仅用于自训练 Word2Vec 词向量，不参与分类任务。

2.3 数据划分

对 NYT 数据集进行随机打乱后，按 80% / 10% / 10% 划分为训练集（9,215 条）、验证集（1,152 条）与测试集（1,152 条）。为保证不同方法间结果可比，全部实验共用同一份划分结果。划分采用固定随机种子（seed=42）并以 `stratify` 分层抽样，确保三个集合中类别比例与原始数据一致，避免某类别被过度分配到单一集合。标签按字母序映射为数字：business=0，politics=1，sports=2。

```
原始数据: (11519, 2)

类别数量: 3

各类别样本数:
label
sports      8639
politics    1451
business    1429
Name: count, dtype: int64

标签 -> 数字 映射: {0: 'business', 1: 'politics', 2: 'sports'}

划分结果:
train: 9215 条
val: 1152 条
test: 1152 条
```

三、评价指标

Accuracy（准确率）：分类正确的样本数占测试样本总数的比例：

$$\text{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}$$

Macro-F1：先对每个类别分别计算 F1 Score，再对所有类别的 F1 取算术平均：

$$\text{F1}_c = \frac{2 \times P_c \times R_c}{P_c + R_c}, \quad \text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} \text{F1}_c$$

其中 P_c（Precision）为该类别预测正确的比例，R_c（Recall）为该类别被正确召回的比例，$C = 3$ 为类别数。由于 Macro-F1 对每个类别等权平均，在类别不均衡的数据上能更全面地反映模型对少数类（politics、business）的分类能力。

四、实验方法与设置

4.1 Task 1：词袋模型（Bag of Words）

使用三种词袋表示，分类器统一为 Logistic Regression（`max_iter=1000`）：

1. **Binary Bag of Words**：文档表示为 $|V|$ 维向量（$|V|$ 为训练集词表大小），第 i 个词出现则为 1，否则为 0，仅考虑词是否出现（`CountVectorizer(binary=True)`）。
2. **Word Frequency**：同样构建 $|V|$ 维向量，但记录每个词的出现次数（`CountVectorizer()`）。
3. **TF-IDF**：词频 × 逆文档频率，对“在几乎所有文档中都出现”的常见词（如 the、a）按其在文档集合中的稀有程度降权（`TfidfVectorizer()`），以突出对类别有区分力的词。

向量化器只在训练集上 fit（学习词表），验证集与测试集仅做 `transform`，避免测试集信息泄漏。词表大小为 60,980。

```
===== Task1-BinaryBoW =====
词表大小: 60980
验证集:  Acc=0.9861   Macro-F1=0.9667
测试集:  Acc=0.9905   Macro-F1=0.9764

===== Task1-WordFreq =====
词表大小: 60980
验证集:  Acc=0.9844   Macro-F1=0.9633
测试集:  Acc=0.9896   Macro-F1=0.9742

===== Task1-TFIDF =====
词表大小: 60980
验证集:  Acc=0.9800   Macro-F1=0.9568
测试集:  Acc=0.9887   Macro-F1=0.9737

结果已写入 results.csv
```

4.2 Task 2：词向量（Word Embedding / Word2Vec）

统一使用 100 维词向量，文档向量由公式 $\mathbf{e}(d) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{e}(w_i)$ 计算，即对文档内所有可查到的词向量取平均（mean pooling），查不到的单词（OOV）跳过。分类器仍为 Logistic Regression。三组实验仅词向量来源不同：

1. **预训练 GloVe**：使用 glove.6B 的 100 维词向量（词表 40 万词）。其训练语料为 Wikipedia 2014 与 Gigaword 5（含大量新闻文本），与新闻领域天然对口。
2. **AG News 自训练 Word2Vec**：用 90,000 条 AG News 文本训练。参数：`vector_size=100, window=5, min_count=5, epochs=5`，学得 22,504 个词。
3. **NYT 自训练 Word2Vec**：用 NYT **训练集**（9,215 条）训练，参数同上，学得 27,099 个词。仅使用训练集训练词向量，防止测试集文本信息泄漏。

分词采用 gensim 的 `simple_preprocess`（转小写并按标点切词）。

```
========== 实验 2-1: 预训练 GloVe ==========
找到本地 GloVe 文件，直接加载
  GloVe 词表大小: 400000
  NYT 训练集词汇覆盖率: 99.8%

===== Task2-GloVe =====
验证集:  Acc=0.9766   Macro-F1=0.9477
测试集:  Acc=0.9861   Macro-F1=0.9669

========== 实验 2-2: AG News 自训练 Word2Vec ==========
ag.csv 形状: (90000, 1)，列名: ['text']
  对 AG News 分词中...
  完成，用时 3.8s
  训练 Word2Vec: 90000 篇文档，100 维，5 个 epoch...
  完成，用时 7.2s，学到 22504 个词
  NYT 训练集词汇覆盖率: 95.9%

===== Task2-W2V-AGNews =====
验证集:  Acc=0.9714   Macro-F1=0.9377
测试集:  Acc=0.9783   Macro-F1=0.9470

========== 实验 2-3: NYT 自训练 Word2Vec ==========
  训练 Word2Vec: 9215 篇文档，100 维，5 个 epoch...
  完成，用时 15.2s，学到 27099 个词
  NYT 训练集词汇覆盖率: 99.0%

===== Task2-W2V-NYT =====
验证集:  Acc=0.9800   Macro-F1=0.9547
测试集:  Acc=0.9835   Macro-F1=0.9580

全部完成，结果已追加写入 results.csv
```

（所示结果为 gensim 分词）

4.3 Task 3：BERT 微调

使用预训练模型 `google-bert/bert-base-uncased`（约 1.1 亿参数），在其顶部添加随机初始化的 3 类分类头，随后在 NYT 训练集上微调全部参数。设置：`max_length=64`（超长截断、不足补零），`epochs=3`，batch size=32，学习率 2×10^{-5}，优化器 AdamW。设备为 NVIDIA GeForce RTX 5060 Laptop GPU，每 epoch 约 53 秒。

```
开始训练: 3 epochs, 每 epoch 288 步
  epoch 1, step 100/288, 当前 loss 0.0387
  epoch 1, step 200/288, 当前 loss 0.1721
epoch 1 完成, 平均 loss 0.1755, 验证集 Acc=0.9774 Macro-F1=0.9524, 用时 53.2s
  epoch 2, step 100/288, 当前 loss 0.0456
  epoch 2, step 200/288, 当前 loss 0.0355
epoch 2 完成, 平均 loss 0.0446, 验证集 Acc=0.9748 Macro-F1=0.9447, 用时 52.7s
  epoch 3, step 100/288, 当前 loss 0.0059
  epoch 3, step 200/288, 当前 loss 0.0566
epoch 3 完成, 平均 loss 0.0205, 验证集 Acc=0.9766 Macro-F1=0.9524, 用时 52.6s

===== Task3-BERT =====
测试集:  Acc=0.9748   Macro-F1=0.9459
Writing model shards: 100%|                                                  | 1/1 [00:01<00:00,  1.53s/it]
模型已保存到 bert_nyt/，结果已写入 results.csv
```

五、实验结果

全部 7 组实验在 NYT Test Set（1,152 条）上的结果汇总如下：

方法	词向量/特征来源	Accuracy	Macro-F1

Binary BoW	词表（出现与否）	**0.9905**	**0.9764**
Word Frequency	词表（出现次数）	0.9896	0.9742
TF-IDF	词表（加权）	0.9887	0.9737
GloVe (100d)	预训练，40 万词	0.9861	0.9669
Word2Vec (AG News)	自训练，22,504 词	0.9783	0.9470
Word2Vec (NYT)	自训练，27,099 词	0.9835	0.9580
BERT-base-uncased	微调	0.9748	0.9459

六、结果分析

（1）三种词袋方法内部比较。 三种表示差距很小（Acc 均在 98.9% 以上），TF-IDF 未表现出明显优势。原因在于新闻类别区分主要依赖领域关键词（如“senator”“match”“stock”），高频常见词虽多但几乎不跨类别重叠，TF-IDF 降权的收益因此有限。

（2）词袋 vs 词向量。 三组词向量方法整体略低于词袋模型。mean pooling 将一篇长新闻压缩为单一向量，丢失了词序信息与关键词的独立信号；而词袋模型在高维稀疏空间（60,980 维）中保留每个词的独立位置，配合逻辑回归的逐词权重，天然适合“靠关键词区分”的短距分类任务。此外新闻文本较长，平均操作会稀释关键信号。

（3）三组词向量内部比较。 GloVe 最优、NYT 自训练居中、AG News 自训练最差，与训练语料规模和领域对口程度完全一致：GloVe 词表最大（40 万）且训练语料包含 Gigaword 新闻文本，对 NYT 训练集词汇覆盖率高达 99.8%；NYT 自训练语料最对口（覆盖率 99.0%）但规模仅 9,215 篇，词表偏小；AG News 语料虽达 9 万条但文本短、话题少，覆盖率仅 95.9%。这验证了词向量质量由“语料规模 × 领域匹配度”共同决定的规律。

（4）BERT 的表现。 BERT 的 Acc（97.5%）与 Macro-F1（94.6%）低于词袋模型，但并不意味着 BERT 方法失败：其一，训练样本仅 9,215 条，远不足以充分微调 1.1 亿参数的大模型；其二，`max_length=64` 截断了大部分新闻正文，模型实际只看到新闻开头；其三，本数据集三类话题可分性极强，词袋 + 逻辑回归已接近性能上限（99%），大模型的表示能力无从发挥。BERT 的优势应在更大规模、类别更细粒度、语义更隐晦的数据上体现。验证集上三 epoch 的 Acc（97.7/97.5/97.7）稳定，说明训练收敛且未过拟合。

（5）Accuracy 与 Macro-F1 的关系。 各组实验中 Macro-F1 始终低于 Accuracy，且差距（约 1.4~2.9 个百分点）与类别不均衡程度相符——这正是少数类（politics、business）分类误差被 Macro-F1 放大的体现，印证了不均衡数据下必须同时报告两个指标的必要性。

七、结论

本实验完成了 7 组文本分类对照实验。主要结论如下：（1）在该规模与类别可分性条件下，词袋模型 + 逻辑回归是性价比最高且效果最好的基线；（2）词向量质量由训练语料规模与领域匹配度共同决定，外部大规模预训练向量（GloVe）优于小规模自训练向量；（3）BERT 微调流程实现正确、训练稳定，但其优势在数据规模受限、任务难度较低的场景下难以体现；（4）类别不均衡时 Macro-F1 是比 Accuracy 更全面的评价指标。

八、复现说明

代码仓库包含数据划分脚本、三个任务的实验脚本与运行说明（README.md）。环境依赖：Python 3、pandas、numpy、scikit-learn、nltk/gensim、torch、transformers。运行顺序：`datasplit.py` → `task1.py` → `task2.py` → `task3.py`，各实验输出统一追加至 `results.csv`。